MoveInSync Backend Case Study 2026

SDE – Java Backend Hiring Assessment · Advanced Track (IIIT & NIT) · Case Study 5

Case Study 5: HR Roster Sync – Idempotent, Out-of-Order, Self-Healing

Corporate clients keep their employee list in an HR system and push changes to MoveInSync: new joiners, transfers, shift changes, exits. Changes arrive through an API in near real time, and a full nightly export of up to 200,000 employees arrives as a file. In practice updates arrive twice, arrive out of order, contain bad data, and are sometimes cut short. Some fields belong to HR and some belong to the transport admin, such as an employee's preferred pickup point. Your job is to keep MoveInSync's employee records correct anyway.

I. Ingest

- Events — create, update and deactivate events, each with the source system's version or timestamp.
- Validate at the Boundary — reject invalid records (a malformed phone number, an unknown office code) with a per-record error report. One bad record must never fail a whole batch.

II. Ordering, Idempotency & Ownership

- Per-Employee Order — apply changes per employee in version order. A stale update is ignored, and a duplicate is a no-op.
- Field Ownership — HR owns name, email, manager and office. The transport admin owns pickup point and special-assistance needs. An HR update never overwrites a transport-owned field, and the conflict rule is written down.

III. Nightly Reconciliation

- Efficient Diff — compare the 200,000-row export with your store without loading both fully into memory, for example with a hash per record and hashed buckets.
- Safety Brake — if tonight's export is missing 30% of active employees, it is almost certainly a broken export. Stop, alert an operator, and deactivate nobody until a human approves.

IV. Downstream Effects

- Reliable Change Events — publish a change event for every applied change using the outbox pattern, so the database update and the event can never disagree.
- Deactivation Side Effects — an employee who leaves has their future cab bookings cancelled and their seat freed.

Example Use Case

• An office transfer (version 7) arrives before the shift change (version 6). The shift change is ignored as stale and the employee ends up in the new office with the version 7 shift.

• The nightly file is cut off at 120,000 rows. The safety brake stops the run, nobody is deactivated, and the operator sees how many records the file appears to be missing.

Key Points to Keep in Mind

- Define the idempotency key and the version comparison. Show a test that replays the same feed twice and changes nothing.
- The safety brake is the difference between a bad night and 80,000 employees without a cab. Make its threshold configurable.
- Never write personal data into logs or error reports. Use employee IDs.
- **Prove it:** diff a 200,000-row snapshot in under 60 seconds with bounded memory, and run a test that crashes the process between the database write and the event publish.

Plus Points in Implementation (Overall Evaluation Criteria)

1. **Authentication:** Implement robust authentication and role-based access (employee vs admin vs operator), with tenant isolation so one client can never read another client's data.
2. **Cost Estimation – Time and Space:** Analyse the time and space complexity of your core logic, and back it with a small benchmark. State where the system stops scaling and why.

3. **Handling System Failure Cases:** Implement fault-tolerant mechanisms for crashes, timeouts, duplicate and out-of-order messages, and partial failures. Show at least one failure-injection test.
4. **Object-Oriented Programming (OOPS):** Choose a robust OOPS language (Java preferred). Apply encapsulation, inheritance and polymorphism for maintainable, extensible code with clean controller / service / repository layering.
5. **Trade-offs in the System:** Clearly define and document the trade-offs you made, especially consistency vs availability vs latency. Explain what you deliberately left out.
6. **System Monitoring:** Add metrics and structured logging. Expose the counters an on-call engineer would need at 3 AM (lag, error rate, rejections, retries, queue depth).
7. **Caching:** Integrate caching for hot data with a sensible eviction policy. Say exactly what can become stale, for how long, and how staleness is bounded.
8. **Error and Exception Handling:** Build a robust error-handling framework with meaningful, non-leaking error messages. No personal data (phone, email, home location) in logs or error responses.

What “Advanced Track” Means

- **Proven guarantees** – every hard rule in your case study has an automated test, including at least one concurrency test and one failure test.
- **Measured performance** – a small load test or benchmark with throughput and p50 / p99 latency, reported in the README with the machine you ran it on.
- **Design note** – one or two pages with an architecture diagram, one sequence diagram for the hardest flow, and a failure-mode table (what fails, what the user sees, how the system recovers).
- **One-command run** – the project starts with a single command (for example docker compose up), including any database or message broker it needs.

Instructions

1. **Read and Understand the Problem Statement:** Carefully read the case study. Understand the requirements, inputs, expected outputs, the hard rules and the scale targets.
2. **Choose a Programming Language:** Java with Spring Boot is preferred. C#, Go, Kotlin or Python are acceptable if you can defend the choice.
3. **Design Your Solution:** Plan first. Decide your data model, the main components, the APIs and the failure modes before writing code. Put this in the design note.
4. **Write the Code:** Write clean, layered code with meaningful names. Keep the core algorithm in plain classes that can be tested without a web server or database.
5. **Test Your Code:** Test normal inputs AND edge cases. Add unit tests for the core logic, plus the concurrency, failure and benchmark tests described in your case study.
6. **Document Your Code:** Add a README with how to run it, your assumptions, the benchmark results, and a short note on the trade-offs you made.
7. **Submit Your Solution:** Push to a public GitHub repository and share the link. Keep a meaningful commit history, not one giant commit.
8. **Demonstration:** Add a short demo video showing the main flows, one failure scenario being handled live, and a few sample API calls (Postman / curl).